

AI Data Analytics Agent

An Enterprise-Grade, Multi-Agent Intelligent Business Intelligence and RAG platform with Automated Profiling, Cleaning, SQL Generation, and Document Compiler Pipelines.

Document Version:	1.2.0 (Stable Release)
Author:	Advanced Agentic BI Engineering Group
Target Audience:	Systems Architects, Interns, Technical Directors
Compilation Date:	2026-06-19
Active Context Workspace:	DuckDB & SQLite Persistent Storage

Table of Contents & Executive Summary

Table of Contents Reference Guide

• Chapter 1: Introduction to Agentic Business Intelligence Systems	Page 3
• Chapter 2: Detailed Architectural Tech Stack Analysis	Pages 4-7
• Chapter 3: Repository Layout & Structural Philosophy	Pages 8-9
• Chapter 4: Tabular and Document File Ingestion Pipelines	Page 10
• Chapter 5: Advanced Data Quality Diagnostics & Recommendations Heuristics	Pages 11-12
• Chapter 6: High-Performance Data Cleaning, Imputation & Derived KPI Recovery	Pages 13-15
• Chapter 7: SQL Query Generation, OLAP DuckDB Execution, and Safety Guards	Pages 16-17
• Chapter 8: Unstructured Text Parsing, Vector Embeddings & Document RAG Engine	Pages 18-19
• Chapter 9: Visual Chart Analytics Recommendations & Headless Chart Exporter	Page 20
• Chapter 10: Task Scheduling, Cron Executions, and Resilience State	Page 21
• Chapter 11: Document Export Wizard Architecture (PDF, PPTX, DOCX, HTML)	Page 22
• Chapter 12: React Frontend Dashboard State Coordination & Interface Design	Pages 23-24
• Appendix: Directory Repository Code Files & Operational Dictionary Map	Pages 25-27

Executive Summary

The AI Data Analytics Agent platform represents an enterprise-grade solution for automatic, semantic, and quantitative data exploration. Traditional business intelligence (BI) requires manual SQL engineering, rigid visualization mapping, and static pipeline adjustments. This system automates this entire lifecycle by linking multi-agent systems. When a user uploads any file, the system profiles its quality, recommends cleaning configurations, compiles a sandbox environment inside DuckDB, generates SQL statements from conversational text, draws interactive Plotly configurations, outputs cited answers for text document uploads via a FAISS vector index, and generates reports. The goal is to provide unified analytics and document summaries in one application.

Chapter 1: Introduction to Agentic Business Intelligence

The Paradigm Shift in Business Analytics

Modern enterprises compile large amounts of data daily, but extracting actionable value remains a bottleneck. Business analytics typically requires a data analyst to write SQL queries, extract reports, and assemble presentation slides. This manual loop delays decision-making.

Open-Agent BI Architectures

Agentic architectures delegate discrete processing tasks to specialized sub-agents that collaborate together. Rather than using a single model to do everything, the system splits responsibilities among multiple modules:

1. **Quality Ingestion Agent:** Inspects dataset anomalies and establishes data quality scores.
2. **Data Cleaning Agent:** Cleans, standardizes categories, imputes missing values, and handles outliers.
3. **SQL Code Generation Agent:** Builds SQL statements mapping schema keys to natural text.
4. **Insights Translation Agent:** Translates database metrics into plain-English summaries.
5. **RAG Assistant Agent:** Indexes semantic text files for vector-based search.
6. **Visualization Agent:** Pairs dataset headers with matching graphical templates.
7. **Task Automation Agent:** Runs background scheduling and compiles scheduled reports.
8. **Document Compiler Agent:** Exports analyses as formatted PDF, PPTX, or Word documents.

This system architecture isolates failure domains and enables rule-based fallbacks. When Gemini exceeds limits or goes offline, rule-based algorithms execute SQL queries and summarize stats to ensure continuity.

Chapter 2: Detailed Architectural Tech Stack Analysis

Part I: Core Backend Framework & Web Servers

The backend is built with FastAPI, which uses ASGI (Asynchronous Server Gateway Interface) standards to handle concurrent requests asynchronously. This design allows the system to process large data files, run LLM prompts, query databases, and write Excel spreadsheets in the background without blocking server threads.

FastAPI Routing and Pydantic Schema Parsing

FastAPI uses Pydantic schemas to validate JSON payloads sent to routes. This ensures type safety at API endpoints. For example, the cleaning endpoint validates requests using Pydantic classes:

```
class CleanRequest(BaseModel):  
    approved_actions: List[str]
```

Web Server Coordination via Uvicorn

Uvicorn runs the FastAPI application. It is configured to monitor and reload API modules during updates, while utilizing CORS middleware to manage client requests safely:

```
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=["*"],  
    allow_credentials=True,  
    allow_methods=["*"],  
    allow_headers=["*"],  
)
```

Metadata Persistence via SQLite3

While datasets are stored elsewhere, SQLite (app.db) serves as the persistent metadata catalog. It tracks workspaces, dataset versions, chat threads, scheduler parameters, and export reports in relational tables using raw SQL queries.

Chapter 2: Detailed Architectural Tech Stack Analysis (Cont.)

Part II: Data Engineering & High-Performance OLAP Engine

At the core of the data analytics pipeline is DuckDB, an embedded, column-store relational database engine optimized for analytical queries (OLAP). Traditional databases like SQLite process row-by-row, which is slow for aggregations on millions of rows. DuckDB uses columnar storage and vectorized execution, keeping processing overhead low during data scans.

Pandas Integration & Connection Architecture

DuckDB integrates directly with Pandas DataFrames. It can query Pandas objects directly in memory without serializing them, speeding up data transfers. In our implementation, `DuckDBManager` manages database connection lifecycles:

```
class DuckDBManager:
    def __init__(self):
        root_dir = Path(__file__).resolve().parent.parent.parent
        self.db_path = str(root_dir / "analytics.duckdb")

    def get_connection(self):
        return duckdb.connect(self.db_path)
```

DataFrame Storage and CSV Import Fallbacks

When saving data, `DuckDBManager` first registers the Pandas DataFrame as a virtual table. If in-memory registration fails, it falls back to writing a temporary CSV and importing it via `read\_csv\_auto`:

```
conn.register("temp_df", dataframe)
conn.execute(f"CREATE OR REPLACE TABLE {table_name} AS SELECT * FROM temp_df")
```

Chapter 2: Detailed Architectural Tech Stack Analysis (Cont.)

Part III: Semantic Search Indexing & LLM Foundations

For unstructured document uploads (PDF, DOCX, TXT), standard SQL queries are not applicable. Instead, the system uses semantic search and Retrieval-Augmented Generation (RAG) to locate relevant information and answer qualitative questions.

Sentence Transformers for Semantic Embeddings

The RAG engine uses the `all-MiniLM-L6-v2` model from Sentence Transformers. It converts text paragraphs into 384-dimensional dense vector embeddings. These vectors capture the semantic meaning of the text, allowing similarity calculations instead of simple keyword matching.

FAISS Vector Indexing & Search Mechanics

Once embeddings are generated, FAISS (Facebook AI Similarity Search) creates an `IndexFlatL2` vector index. FAISS calculates the Euclidean distance between the query vector and document index vectors to quickly retrieve the top 3 (K) most similar text paragraphs:

```
index = faiss.IndexFlatL2(dim)
index.add(embeddings)
D, I = index.search(query_embedding, top_k)
```

Google Gemini API Client Integration

The system uses `gemini-2.5-flash` for language tasks. `gemini\_client.py` initializes the Google GenAI SDK using `GOOGLE\_API\_KEY` loaded from `.env`. The model processes prompts for SQL generation, data summaries, and document Q&A.

Chapter 2: Detailed Architectural Tech Stack Analysis (Cont.)

Part IV: Reporting Engines and Frontend Dashboard Tech Stack

To support business operations, the platform exports insights into office-compatible formats. It uses three libraries to compile structured layouts programmatically from dataset tables and chat streams.

ReportLab, python-pptx, and python-docx

- **ReportLab:** Compiles PDF documents. It uses `SimpleDocTemplate` flowables alongside a two-pass `canvas` callback (`NumberedCanvas`) to calculate exact page counts and draw headers/footers.
- **python-pptx:** Assembles slide decks. It maps template placeholders to left and right pane configurations (SQL text left, chart graphics right).
- **python-docx:** Compiles Word documents. It formats text, inserts code snippets in Courier New, and embeds images.

Headless Matplotlib Engine

Matplotlib converts interactive browser charts into static PNG files for export. By setting `matplotlib.use('Agg')`, it runs in headless mode without a GUI, saving charts as temporary files before deleting them:

```
import matplotlib
matplotlib.use('Agg') # Enable headless execution
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7, 4.2), dpi=300)
```

Frontend Framework & Plotly.js Visualizations

The React frontend uses Vite for fast building and HMR. It includes Plotly.js to render interactive charts (zoom, pan, png downloads) using configuration JSON schemas dynamically built by the backend.

Chapter 3: Repository Layout & Structural Philosophy

Backend Directory Tree & Structure Blueprint

The workspace isolates frontend client files from backend processing modules. Below is a detailed view of the backend directory structure:

```

backend/
  agents/
    cleaning_agent/      # Data cleaning engine, validations
    code_gen_agent/      # LLM SQL translation modules
    insight_agent/       # Analysis summary modules
    visualization_agent/ # Chart formatting, matplotlib exports
    quality_agent.py     # Main diagnostics profiling agent
    rag_agent.py         # Vector search, paragraph chunking
    scheduler_agent.py   # APScheduler background runner
  api/
    upload_routes.py     # Raw file ingestion and loader factory
    chat_routes.py       # Q&A messaging routes, backups
    cleaning_routes.py   # Interactive cleaning actions commit
    workspace_routes.py  # Duplication, deletions, rollbacks
    export_routes.py     # PDF, DOCX, PPTX builders
    scheduler_routes.py  # Task schedule config routes
  database/
    duckdb_manager.py    # Local Storage Gateways
    duckdb_manager.py    # duckdb_manager class (analytics.duckdb)
    sqlite_manager.py    # sqlite_manager class (app.db schema)
  file_processing/
    parser_factory.py    # Standard Parsers
    parser_factory.py    # Extension factory mapping
    pdf_loader.py        # pdfplumber text & table parsing
    csv_loader.py, docx_loader.py, tsv_loader.py, excel_loader.py...
  llm/
    gemini_client.py     # Gemini API Client Wrapper
    gemini_client.py     # SDK config, connection states
    prompts.py           # Core prompts
  main.py               # FastAPI application main route

```


Chapter 3: Repository Layout & Structural Philosophy (Cont.)

Frontend Directory Tree & Component Architecture

The React dashboard is organized into components to manage sidebar, chat, results, and upload workflows:

```
frontend/
├── src/
│   ├── components/           # Independent React Widgets
│   │   ├── ChartView.jsx     # Plotly.js chart component
│   │   ├── ChatPanel.jsx     # Message log thread controller
│   │   ├── ResultTable.jsx   # Query output table grid
│   │   ├── SqlViewer.jsx     # Syntax highlighter widget
│   │   ├── UploadPanel.jsx   # Dropzone file loader
│   │   └── WorkspaceList.jsx # Workspace listing sidebar widget
│   ├── services/             # Network Clients
│   │   └── api.js            # Axios API wrappers
│   ├── App.css               # Glassmorphism tokens stylesheet
│   ├── index.css             # Styling theme tokens variables
│   ├── App.jsx               # Main coordinate state wrapper
│   ├── main.jsx              # React root mounting wrapper
│   ├── index.html            # HTML shell entry point
│   └── package.json          # npm package dependency manifest
```

Design System Integration

The CSS system uses dark Apple-inspired minimalist styles. The stylesheet defines color variables, translucent backgrounds using `backdrop-filter: blur(20px)`, and subtle borders. Focus effects use violet highlights (`box-shadow: 0 0 30px rgba(167,139,250,0.15)`) to indicate active states.

Chapter 4: Tabular and Document File Ingestion Pipelines

Ingestion Architecture & Routing Flow

File uploads are received at the `/upload/` route. The system maps raw file buffers to their respective parser loaders using `ParserFactory`:

```
def parse(file_path: str):
    extension = Path(file_path).suffix.lower()
    if extension == ".csv":
        return CSVLoader.load(file_path)
    elif extension in [".xlsx", ".xls"]:
        return ExcelLoader.load(file_path)
    elif extension == ".pdf":
        return {"text": PDFLoader.extract_text(file_path),
                "tables": PDFLoader.extract_tables(file_path)}
```

Tabular Ingestion Pipeline

For structured datasets (CSV, Excel, TSV, Parquet), `upload\_routes.py` reads files into a Pandas DataFrame. It standardizes column names (lowercase, space-to-underscore), applies basic cleaning steps, generates schema/metadata logs, creates a SQLite workspace, writes Version 1 to `uploads/cleaned/`, and registers the DataFrame as a table in DuckDB.

Document Ingestion Pipeline

For text files (PDF, Word, TXT), `pdf\_loader.py` uses `pdfplumber` to extract raw text and reconstruct tables page-by-page. Extracted tables are saved as DuckDB tables, and document text is stored in SQLite for RAG vector search.

Chapter 5: Advanced Data Quality Diagnostics & Heuristics

Diagnostics Profiling Metrics

When a workspace is loaded, `QualityAgent` profiles the dataset. It runs diagnostics to generate a data quality score (0 to 100%) and build recommended cleaning actions:

1. **Duplicate Rows:** Scans for duplicate rows and calculates duplication ratios.
2. **Near-Duplicates:** Compares rows for 90%+ similarity, ignoring unique keys and dates, to find duplicated records.
3. **Null Counts:** Counts empty cells per column. Columns that are entirely empty subtract 10 points from the quality score.
4. **Mixed Capitalization:** Scans string columns for lowercase/uppercase inconsistencies (e.g. 'Male', 'male', 'MALE').
5. **Invalid Formats:** Scans for currency symbols or commas stored in numeric columns, suggesting conversion to floats.

Levenshtein Category Near-Duplicate Checks

To detect near-duplicate category names (e.g. 'Apple Inc.' vs 'Apple Inc'), `QualityAgent` calculates the Levenshtein distance between unique string values:

```
def levenshtein(s1, s2):
    if len(s1) < len(s2): return levenshtein(s2, s1)
    if len(s2) == 0: return len(s1)
    previous_row = range(len(s2) + 1)
    for i, c1 in enumerate(s1):
        current_row = [i + 1]
        for j, c2 in enumerate(s2):
            insertions = previous_row[j + 1] + 1
            deletions = current_row[j] + 1
            substitutions = previous_row[j] + (c1 != c2)
            current_row.append(min(insertions, deletions, substitutions))
        previous_row = current_row
    return previous_row[-1]
```

Chapter 5: Advanced Data Quality Diagnostics & Heuristics (Cont.)

Numerical Outlier & Impossible Boundary Checks

`QualityAgent` runs statistical checks on numerical columns to identify outliers and impossible values:

Interquartile Range (IQR) Outlier Checks

The system calculates the Interquartile Range ($IQR = Q3 - Q1$). Values outside the IQR bounds (using a threshold of $3.0 * IQR$) are flagged as outliers:

```
q1 = col_series.quantile(0.25)
q3 = col_series.quantile(0.75)
iqr = q3 - q1
lower_bound = q1 - 3.0 * iqr
upper_bound = q3 + 3.0 * iqr
outliers = (col_series < lower_bound) | (col_series > upper_bound)
```

Z-Score Outlier Checks

Z-score metrics identify values more than 3 standard deviations from the column mean, capturing anomalies in normally distributed data:

```
z_scores = (col_series - mean) / std
z_outliers = z_scores.abs() > 3
```

Impossible Values Diagnostics

The system alerts users if pricing, salary, or quantity columns contain negative numbers, or if age columns contain values outside the range of [0, 120]. These errors are flagged for review or automated nulling during data cleaning.

Chapter 6: High-Performance Data Cleaning Heuristics

Standard vs Aggressive Cleaning Modes

`DataCleaner` executes cleaning operations. When running, the cleaner applies basic structural improvements automatically, while major changes require user approval via the API.

Core Structural Transformations

These default operations run on every raw upload without requiring manual confirmation:

- **Casing Standardization:** String column values are formatted using Title Case, while abbreviations in a protected set (e.g. ID, SKU, URL) are kept capitalized.
- **Whitespace Stripping:** Removes leading and trailing whitespace from string variables.

```
def _title_case_safe(val: str) -> str:
    words = val.strip().split()
    result = []
    for word in words:
        upper = word.upper()
        if upper in _PROTECTED_UPPERCASE: result.append(upper)
        else: result.append(word.capitalize())
    return " ".join(result)
```

- **Boolean Normalization:** Converts text inputs (yes, no, true, false, 1, 0) into standard boolean types.
- **Gender Standardizing:** Standardizes input categories (e.g. M, Male, F, Female) into 'Male' or 'Female'.
- **Currency Formatting Parser:** Strips currency symbols (\$, €, £) and commas, converting numeric text columns into floats/ints.

Chapter 6: High-Performance Data Cleaning Heuristics (Cont.)

User-Approved Cleaning Operations

When the user selects diagnostic recommendations on the dashboard, `clean\_workspace` calls the API with the selected action IDs. The backend applies these changes to the raw data and creates a new dataset version.

1. Imputing Missing Values

For text categorical columns, missing values are imputed as 'Unknown'. For numeric attributes, missing values are replaced with the column median to preserve statistical distributions:

```
if "impute_missing" in action_id:
    if role == "DIMENSION":
        df[col] = df[col].fillna("Unknown")
    elif role == "ATTRIBUTE":
        median_val = df[col].median()
        df[col] = df[col].fillna(median_val)
```

2. Flagging Outliers

Rather than deleting rows containing outliers, the system flags them by setting a boolean column `is\_outlier` to True. This keeps the data available while letting the query engine filter outliers if needed.

3. Spelling Category Consolidation

For dimension columns, the system merges values with similar spelling (determined by Levenshtein distance thresholds) to clean up categorical groupings.

4. Cleaning Invalid Values

Replaces values that violate business rules (e.g. negative prices, unparseable dates) with Null, allowing subsequent imputation or clean exclusions.

Chapter 6: High-Performance Data Cleaning Heuristics (Cont.)

Derived Metric Recovery & Integrity Auditing

The system includes derived metric calculations and data integrity validations to ensure consistency after cleaning operations.

Derived Metric Recovery Algorithms

`DataCleaner.recover_derived_metrics`` recovers missing values in derived columns when their component values are present. For example, if a row contains `units_sold`` and `unit_price`` but lacks a `revenue`` value, it calculates the missing value automatically:

```
# Recovery math for missing values
if df['revenue'].isna() and df['units_sold'].notna() and df['unit_price'].notna():
    df['revenue'] = df['units_sold'] * df['unit_price']
if df['profit'].isna() and df['revenue'].notna() and df['cogs'].notna():
    df['profit'] = df['revenue'] - df['cogs']
```

Analytics Integrity Audits

Before committing cleaning changes, `run_analytics_integrity_check`` runs metrics audits. It compares summary values (e.g., total revenue, profit, units sold, row counts) before and after cleaning. If a change exceeds a 1% threshold, the system flags a warning requiring user confirmation:

```
row_change = abs(after_rows - before_rows) / max(before_rows, 1)
if row_change > 0.01:
    warning_triggered = True
    warnings.append("Row count changed significantly.")
```

Chapter 7: SQL Query Agent & DuckDB Execution Engine

Part I: SQL Code Generation Pipeline

For structured datasets, natural language questions are converted into SQL queries using Gemini prompt structures. The generated SQL runs on the DuckDB database engine.

Gemini SQL Generation Prompts

The `SQL\_GENERATION\_PROMPT` template formats database schemas, columns, table names, and user questions. It instructs the LLM to output valid DuckDB-compliant SQL queries and returns a plain string containing only the query.

Rule-Based Backup Fallback Engine

If Gemini is offline, the system falls back to a rule-based SQL generator. It parses key terms (e.g. 'sum', 'total', 'average', 'count') and schema column names using regex patterns to construct standard SQL queries:

```
def simple_sql_fallback(question, table, schema):
    ql = question.lower()
    numeric_cols = [c for c, v in schema.items() if "int" in v.get("dtype", "") or "float" in v.get("dtype", "")]
    if "total" in ql or "sum" in ql:
        target = "sales" if "sales" in schema else numeric_cols[0]
        return f"SELECT SUM({target}) AS total_{target} FROM {table}"
    if "count" in ql or "how many" in ql:
        return f"SELECT COUNT(*) AS row_count FROM {table}"
    return f"SELECT * FROM {table} LIMIT 10"
```


Chapter 7: SQL Query Agent & DuckDB Execution Engine (Cont.)

Part II: SQL Execution, Safety Gates, & KPI Footnotes

After a SQL query is generated, the system checks it against safety constraints before executing it on DuckDB and formatting the output.

Query Safety Interceptors

To prevent data modification or directory access, the database execution layer blocks queries containing write or structural keywords, raising a security exception if found:

```
forbidden_keywords = ["delete", "drop", "alter", "update", "insert"]
for kw in forbidden_keywords:
    if re.search(r"\b" + re.escape(kw) + r"\b", query_clean):
        raise ValueError("Unauthorized SQL operation detected.")
```

Data Quality KPI Footnotes

To track missing values, `QueryAgent` calculates completeness statistics for grouping columns and metric columns (like revenue and quantity). It appends footnotes to insights, detailing the percentage of valid data and highlighting missing metrics:

```
# Dimension completeness calculation
comp_pct = (valid_rows / total_rows) * 100
completeness_msg = f"Dimension Completeness: {comp_pct:.1f}% of rows contain a valid dimension value.
"
if missing_rows > 0:
    footnote = f"Data Quality Notice: {missing_rows} rows could not be attributed due to missing values."
    answer += "\n\n" + completeness_msg + "\n" + footnote
```

Chapter 8: Unstructured Text Parsing & Document RAG Engine

Part I: Document Indexing & Embedding Pipelines

For document uploads (PDF, DOCX, TXT), the system builds a vector search index using SentenceTransformers and FAISS, allowing retrieval of relevant text blocks.

Chunking and Embedding Text

The system chunks document text by double newlines (paragraphs) or single newlines to preserve context. Each chunk is processed by the embedding model to generate a 384-dimensional vector:

```
model = SentenceTransformer("all-MiniLM-L6-v2")
docs = [p.strip() for p in text.split("\n\n") if p.strip()]
embeddings = model.encode(docs, convert_to_numpy=True)
```

Building the FAISS Vector Index

The generated vector embeddings are loaded into a FAISS L2 flat index, which is saved to the workspace folder ('rag_indexes/') along with the raw text blocks for retrieval:

```
dim = embeddings.shape[1]
index = faiss.IndexFlatL2(dim)
index.add(embeddings)
faiss.write_index(index, "rag_indexes/ws_1.index")
# Save document blocks text
with open("rag_indexes/ws_1.docs", "w") as f:
    for d in docs: f.write(d + "\n--\n")
```

Chapter 8: Unstructured Text Parsing & Document RAG Engine (Cont.)

Part II: Similarity Retrievals, Cited Prompts, and Memory Logs

When a user asks a qualitative question, the system retrieves relevant text blocks and generates a cited answer.

Similarity Searches in FAISS

The search query is embedded using the same vector model. FAISS searches the index to retrieve the index positions of the top 3 most similar text blocks. These positions are used to retrieve the raw text blocks:

```
q_emb = model.encode([query], convert_to_numpy=True)
distances, indices = index.search(q_emb, k=3)
hits = [docs[idx] for idx in indices[0] if idx < len(docs)]
```

Answering with Source Citations

The retrieved text blocks are compiled into a context block. This context is sent to Gemini with instructions to answer the query based only on the excerpts and cite the sources using block markers (e.g. [Block X]):

```
context_str = "\n\n".join([f"[Block {i+1}]: {h}" for i, h in enumerate(hits)])
prompt = f"Answer the question based only on the excerpts.\nContext:\n{context_str}\nQuestion: {query}"
answer = gemini_client.generate(prompt)
```

RAG Chat History Storage

The final answer is saved in the SQLite `chat\_messages` table, preserving the conversation history across workspaces.

Chapter 9: Visual Chart Analytics Recommendations & Headless Exporter

Heuristics Recommendation Engine

`ChartRecommender` inspects query columns and data types to recommend suitable Plotly.js chart configurations for the frontend dashboard:

- **Date + Numeric:** Recommends a line chart, displaying trends over time.
- **Categorical + Numeric:** Recommends a bar chart for categorical metrics.
- **Two Numeric Columns:** Recommends a scatter plot to show correlations.
- **Single Numeric Column:** Recommends a histogram to show distribution frequencies.

Plotly Visual Formatting Specifications

`VisualizationAgent.create\_chart` formats charts using custom layout parameters (transparent backgrounds, dark themes, and standard borders) to match the dashboard's design system.

Headless Matplotlib Chart Exporter

For document exports, the backend uses `ChartExporter` to render high-resolution PNG charts programmatically without a GUI:

```
fig, ax = plt.subplots(figsize=(7, 4.2), dpi=300)
if chart_type == "bar":
    ax.bar(df[x_col], df[y_col], color='#2563eb', alpha=0.85)
elif chart_type == "line":
    ax.plot(df[x_col], df[y_col], marker='o', color='#2563eb')
fig.savefig(out_path, format="png", bbox_inches='tight')
plt.close(fig)
```

Chapter 10: Task Scheduling, Cron Executions, and Resilience

Scheduled Report Architecture & Triggers

The scheduling module allows users to configure automated queries that compile styled Excel reports at set intervals.

Background Execution Management

The system uses `APScheduler` to schedule background jobs. It parses standard frequencies (daily, weekly, monthly) or custom 5-field cron expressions to define job intervals:

```
def add_report_job(self, workspace_id, schedule_id, name, query, frequency, cron_expr=None):
    job_id = f"sch_{schedule_id}"
    trigger_args = {}
    if frequency == "weekly":
        trigger_args = {"trigger": "cron", "day_of_week": "mon", "hour": 9, "minute": 0}
    elif frequency == "cron" and cron_expr:
        parts = cron_expr.split()
        trigger_args = {"trigger": "cron", "minute": parts[0], "hour": parts[1], "day": parts[2], "month": parts[3], "day_of_week": parts[4]}
    self.scheduler.add_job(func=self.run_scheduled_report, args=[workspace_id, schedule_id, name, query], id=job_id, **trigger_args)
```

Scheduled Query Execution

When a job triggers, the query runs on DuckDB. The output dataframe is compiled into an Excel spreadsheet saved under `exports/`, and a record is logged in the SQLite `reports` table.

Startup Reloading

On startup, the system queries active schedules in SQLite and reloads them into the background scheduler to ensure continuity.

Chapter 11: Document Export Wizard Architecture (PDF, PPTX, DOCX, HTML)

Export Architecture & Compilation Pipelines

`export\_routes.py` manages document compilation. It supports both default exports and a customizable wizard that filters content based on user selections.

PDF Compilation Pipeline

The PDF pipeline uses ReportLab to construct the document structure. It formats text sections, builds results tables, and embeds charts generated by the Matplotlib exporter. It uses a custom canvas (`NumberedCanvas`) to handle two-pass page rendering:

```
class NumberedCanvas(canvas.Canvas):
    def showPage(self):
        self._saved_page_states.append(dict(self.__dict__))
        self._startPage()
    def save(self):
        num_pages = len(self._saved_page_states)
        for state in self._saved_page_states:
            self.__dict__.update(state)
            self.draw_page_decorations(num_pages)
            super().showPage()
        super().save()
```

PowerPoint and Word Compilers

- **PowerPoint Compilation:** The slide compiler uses `pptx` layout slide templates. It formats query titles, creates text boxes for insights and SQL scripts, and embeds chart graphics.
- **Word Compilation:** Uses `python-docx` to format reports with sections, Courier New code snippets, tables, and images.

Chapter 12: React Frontend Dashboard State Coordination & Interface Design

State Management & Coordinator Hooks

`App.jsx` serves as the central state coordinator. It fetches the active workspace on launch, maintains active connections, and coordinates data transfers between sub-components.

Workspace Retrieval Hook

The system uses standard React hooks to load workspace data and refresh active panels during updates:

```
const loadActiveWorkspace = async () => {
  try {
    const res = await getActiveWorkspace();
    if (res.workspace) {
      const wsId = res.workspace.id || res.workspace.workspace_id;
      setActiveWorkspace(res.workspace);
      setActiveWorkspaceId(wsId);
      loadProfile(wsId);
      loadVersions(wsId);
      loadReports(wsId);
    }
  } catch (err) { console.error(err); }
};
```

Sidebar Panel Toggles

The sidebar handles panel layout updates, responsive sizing toggles, file uploads, rename/delete requests, and schedules configuration widgets.

Keyboard Shortcut Hooks

React `useEffect` hooks capture keyboard shortcuts (e.g. `Ctrl+K` to search workspaces, `Ctrl+E` to export reports), triggering corresponding dashboard functions.

Chapter 12: React Frontend Dashboard State Coordination (Cont.)

Interface Component Logic & Custom Renderers

Individual component scripts handle rendering logic, syntax highlighting, pagination, and animations:

1. Custom Markdown & SQL Highlighter (*ChatPanel.jsx*)

`ChatPanel` processes markdown responses and formats code blocks. It runs syntax highlighting on SQL snippets by replacing keywords and functions with colored HTML tags:

```
const highlightSql = (sqlText) => {
  let escaped = sqlText.replace(/&/g, "&amp;").replace(/</g, "&lt;");
  const keywords = ["SELECT", "FROM", "WHERE", "GROUP BY", "ORDER BY"];
  keywords.forEach(kw => {
    const regex = new RegExp(`\\b(${kw})\\b`, "gi");
    escaped = escaped.replace(regex, '<span class="text-brand-lime font-bold">$1</span>');
  });
  return <code dangerouslySetInnerHTML={{ __html: escaped }} />;
};
```

2. Results Table Rendering (*ResultTable.jsx*)

`ResultTable` formats and displays DuckDB query results in a paginated grid. It flags null cells and highlights unassigned dimensions.

3. Chart View Renderer (*ChartView.jsx*)

`ChartView` initializes Plotly configurations, applying responsive layout options to match the dashboard's theme.

Appendix: Directory Repository Code Files & Operational Dictionary Map

Primary Backend Code File Responsibilities (Part I)

Below is a functional mapping of files in the workspace, detailing their responsibilities in the data analytics pipeline:

File Path	Functional Responsibility Operations
backend/main.py	Application entrypoint. Initializes FastAPI server, sets up global CORS middleware, includes API routers, and loads active scheduled background jobs.
backend/agents/quality_agent.py	Profiles tabular workspace health. Evaluates null ratios, outlier IQR bounds, capitalization errors, mixed date formats, and runs category near-duplicate checks.
backend/agents/cleaning_agent/cleaner.py	Executes dataset cleaning: standardizes names, converts values to Title Case, imputes missing values, flags outliers, consolidates spellings, and recalculates missing derived metrics.
backend/agents/cleaning_agent/validators.py	Defines structural data validation rules, such as identifying exact duplicate rows or identifying completely blank rows/columns.
backend/agents/code_generation_agent/code_agent.py	Generates DuckDB-compliant SQL statement blocks from conversational text queries using Gemini API models, with a rule-based regex converter fallback.
backend/agents/insight_agent/insight_agent.py	Summarizes database query outputs into plain-text business insights, outlining trends and aggregates.

Appendix (Cont.)

Primary Backend Code File Responsibilities (Part II)

File Path	Functional Responsibility Operations
backend/agents/visualization_agent/visualization_agent.py	Builds Plotly chart layout and trace configurations (bars, scatter, line graphs, histograms, boxes, heatmaps) from query results.
backend/agents/visualization_agent/chart_exporter.py	Uses matplotlib in headless mode to save visualization configurations as local PNG images for document embedding.
backend/agents/rag_agent.py	Splits documents into paragraphs, generates embeddings, builds a FAISS index, and performs similarity searches to compile cited responses.
backend/agents/scheduler_agent.py	Manages background cron jobs. Executes scheduled database queries and compiles them into Excel files.
backend/database/duckdb_manager.py	Manages connections to `analytics.duckdb`. Saves DataFrames and blocks write commands to enforce query safety.
backend/database/sqlite_manager.py	Manages SQLite database operations: registers workspaces, logs chat history, manages version control, and records schedule parameters.
backend/file_processing/parser_factory.py	Acts as the file parser router, matching extensions (CSV, Excel, TSV, Parquet, PDF, DOCX, TXT) to their respective loaders.

Appendix (Cont.)

Primary Backend Routes & Frontend File Responsibilities

File Path	Functional Responsibility Operations
backend/api/upload_routes.py	Receives file uploads, coordinates cleaning actions, builds schema configurations, and saves version metadata.
backend/api/chat_routes.py	Handles query requests and retrieves active conversation logs. Exports chat histories as formatted PDF/Word/TXT files.
backend/api/cleaning_routes.py	Executes dataset cleaning steps and logs update versions in the database.
backend/api/workspace_routes.py	Manages workspaces: lists, duplicates, deletes, and performs database version rollbacks.
backend/api/export_routes.py	Generates custom PDF, PowerPoint, Word, and HTML reports containing query insights and embedded charts.
frontend/src/App.jsx	Main React controller. Manages dashboard state variables, sidebar transitions, modal toggles, and keyboard shortcut triggers.
frontend/src/components/ChatPanel.jsx	Chat window controller. Manages input logs, custom markdown rendering, search filters, and syntax highlights.
frontend/src/services/api.js	Frontend Axios network service. Configures REST client routes to communicate with the FastAPI server.